

Topic 10.2

10.2: Static Members

A **static** data member of a class occurs once for *all* instances of the class, not once for *each* instance.

A **static** function may be invoked as an ordinary function, and does not need to be invoked on an instance of the class. In fact, for static member functions, there is no invoking object, and the *this* pointer is undefined in the body of a static member function.

Static members can often be used for debugging purposes or for instrumenting code (putting in statistics gathering for various purposes including debugging, optimization, etc.).

The following example illustrates the basic mechanics of static member functions and data members.

```
// MODULE:      inherit7.cpp
// PROGRAMMER:  Gary J. Blair
// LANGUAGE:    C++
// DATE:       96/04/11
//
// PURPOSE:
//   Illustrate static data and function
//   members.
//   This program has a bug that illustrates
//   what happens when you forget to declare
//   destructors virtual.
//

#include      <iostream.h>
#include      <stdlib.h>
#include      <string.h>

class Thing  // Thing is the base of
             // all our objects.
{
public:
    Thing() { thing_count++; strcpy(name,""); };
    Thing( Thing& t ) { thing_count++;
        strcpy(name,t.name); };
    Thing( char * thing_name ) { thing_count++;
        strcpy(name,thing_name); };
    ~Thing() { thing_count--; };
    virtual Thing *clone()
        { return new Thing(*this); };
    virtual void print();
    static int ThingCount() { return thing_count; };

protected:
    char name[80];

private:
    static int thing_count;  // Total number
                           // of all things.
};

int Thing::thing_count = 0; // Initialize static
                           // data member.

void Thing::print()
{
```

```

    cout << "name:" << name;
}

class Person : public Thing
{
public:
    Person();
    Person( char *person_name, int person_age );
    Person( Person& p );
    ~Person() { person_count--; };
    virtual Thing *clone()
        { return new Person(*this); };
    void print();
    static int PersonCount() { return person_count; };

protected:
    int age;

private:
    static int person_count;    // Number of
                                // Person objects.
};

int Person::person_count = 0;

Person::Person() : Thing()
{
    person_count++;
    age = -1;
}

Person::Person( char *person_name, int person_age )
    : Thing(person_name)
{
    person_count++;
    age = person_age;
}

Person::Person( Person& p ) : Thing(p.name)
{
    person_count++;
    age = p.age;
};

void Person::print()
{
    Thing::print();
    cout << " age: " << age;
}

void main(void)
{
    Thing  t("Cessna 182");
    Person me("Gary J. Blair",37);
    Thing *t2, *me2;

    cout << "Before cloning # things = "
        << Thing::ThingCount()
        << " # persons = "
        << Person::PersonCount() << endl;
    t2 = t.clone();
    me2 = me.clone();
    cout << "After cloning # things = "
        << Thing::ThingCount()
        << " # persons = "
        << Person::PersonCount() << endl;

    t2->print();
}

```

```

cout << endl;
me2->print();
cout << endl;

delete t2;
delete me2;
cout << "After deleting # things = "
    << Thing::ThingCount()
    << " # persons = "
    << Person::PersonCount() << endl;
}

```

Output of inherit7.cpp:

```

Before cloning # things = 2 # persons = 1
After cloning # things = 4 # persons = 2
name:Cessna 182
name:Gary J. Blair age: 37
After deleting # things = 2 # persons = 2

```

Because I failed to declare my destructor virtual, only the base class destructor is called when each object is destroyed. Thus, the counts for persons are not decremented appropriately. The following version of the program fixes the bug.

```

// MODULE:      inherit8.cpp
// PROGRAMMER:  Gary J. Blair
// LANGUAGE:    C++
// DATE:       96/04/11
//
// PURPOSE:
//   Illustrate static data and function
//   members.

#include <iostream.h>
#include <stdlib.h>
#include <string.h>

class Thing // Thing is the base of
           // all our objects.
{
public:
    Thing() { thing_count++; strcpy(name,""); };
    Thing( Thing& t )
        { thing_count++; strcpy(name,t.name); };
    Thing( char * thing_name )
        { thing_count++; strcpy(name,thing_name); };
    virtual ~Thing() { thing_count--; };
    virtual Thing *clone()
        { return new Thing(*this); };
    virtual void print();
    static int ThingCount() { return thing_count; };

protected:
    char name[80];

private:
    static int thing_count; // Total number
                          // of all things.
};

```

```
int Thing::thing_count = 0; // Initialize static
                          // data member.
```

```
void Thing::print()
{
    cout << "name:" << name;
}
```

```
class Person : public Thing
{
    public:
    Person();
    Person( char *person_name, int person_age );
    Person( Person& p );
    ~Person() { person_count--; };
    virtual Thing *clone()
        { return new Person(*this); };
    void print();
    static int PersonCount()
        { return person_count; };

    protected:
    int age;

    private:
    static int person_count; // Number of
                           // Person objects.
};
```

```
int Person::person_count = 0;
```

```
Person::Person() : Thing()
{
    person_count++;
    age = -1;
}
```

```
Person::Person( char *person_name,
                int person_age )
    : Thing(person_name)
{
    person_count++;
    age = person_age;
}
```

```
Person::Person( Person& p ) : Thing(p.name)
{
    person_count++;
    age = p.age;
};
```

```
void Person::print()
{
    Thing::print();
    cout << " age: " << age;
}
```

```
void main(void)
{
    Thing t("Cessna 182");
    Person me("Gary J. Blair",37);
    Thing *t2, *me2;

    cout << "Before cloning # things = "
         << Thing::ThingCount()
         << " # persons = "
         << Person::PersonCount() << endl;
```

```
t2 = t.clone();
me2 = me.clone();
cout << "After cloning # things = "
    << Thing::ThingCount()
    << " # persons = "
    << Person::PersonCount() << endl;

t2->print();
cout << endl;
me2->print();
cout << endl;

delete t2;
delete me2;
cout << "After deleting # things = "
    << Thing::ThingCount()
    << " # persons = "
    << Person::PersonCount() << endl;
}
```

Output of inherit7.cpp:

```
Before cloning # things = 2 # persons = 1
After cloning # things = 4 # persons = 2
name:Cessna 182
name:Gary J. Blair age: 37
After deleting # things = 2 # persons = 1
```

Topic 10.3 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-10-dot-3>)